

Implement Alpha-Beta Tree search for any game search problem.

```
maximum = 1000
```

```
minimum = -1000
```

```
def fun_alphabeta(d, node, maxp, v, a, b):
```

```
    # Base case: return the value at the leaf node
```

```
    if d == 3:
```

```
        return v[node]
```

```
    if maxp:
```

```
        best = minimum
```

```
        for i in range(0, 2): # loop for the number of child nodes
```

```
            value = fun_alphabeta(d + 1, node * 2 + i, False, v, a, b)
```

```
            best = max(best, value)
```

```
            a = max(a, best)
```

```
            if b <= a:
```

```
                break
```

```
        return best
```

```
    else:
```

```
        best = maximum
```

```
        for i in range(0, 2):
```

```
            value = fun_alphabeta(d + 1, node * 2 + i, True, v, a, b)
```

```
            best = min(best, value)
```

```
            b = min(b, best)
```

```
            if b <= a:
```

```
                break
```

```
        return best
```

```
# Gather inputs
```

```
src = []
```

```
x = int(input("Enter total number of leaf nodes: "))
```

```
for i in range(x):
```

```
    y = int(input("Enter node value: "))
```

```
    src.append(y)
```

```
d = int(input("Enter depth value: "))
```

```
node = int(input("Enter starting node value: "))
```

```
# Compute and print the optimal value
```

```
print("The optimal value is", fun_alphabeta(d, node, True, src, minimum, maximum))
```

Output:

Enter total number of leaf nodes: 8

Enter node value: 1

Enter node value: 2

Enter node value: 3

Enter node value: 4

Enter node value: 5

Enter node value: 6

Enter node value: 7

Enter node value: 8

Enter depth value: 0

Enter starting node value: 0

The optimal value is 6